

IsingFold Project Specification

Quality-first minor embedding with exact validity, typed graph reinforcement learning, and authenticated evaluation

IsingFold Project

12 September 2026

Contents

Purpose and research thesis	1
Objective and validity	2
System architecture	2
State, actions, and work	3
IF-Core model architecture	3
Learning objective and optimization	4
Data preparation and authority	4
Model selection and training	5
Baselines and evaluation	5
Reproducibility and HPC execution	6
Claim boundary	6

Purpose and research thesis

IsingFold studies minor embedding as a quality-aware sequential decision problem. A useful system must return a valid embedding, preserve enough routing freedom to repair difficult states, and improve the quality of the Ising solution obtained after programming and sampling. Minimizing physical qubits or chain length alone is not the target.

The system uses Profile I as its primary experiment. A registered classical initializer supplies a feasible embedding when it can. The learned policy then applies bounded multi-chain rewrites, including controlled temporary overlap, while a protected archive retains a valid fallback. The final embedding is accepted only by an independent validator. Empty-state construction is a separate profile with a different initial distribution and failure constraint.

The central hypothesis is measurable:

Under a sealed population, common deployment selector, explicit resource limits, and matched online wall-clock envelope, sequential learned rewrites improve failure-aware downstream Ising utility beyond the same-support controls and a validation-tuned stock minorminer system.

This is a hypothesis, not a property implied by the architecture. Learned superiority requires the registered three-seed held-out evaluation.

Objective and validity

For logical Ising instance $I = (G_L, h, J)$,

$$E_I(s) = \sum_i h_i s_i + \sum_{(i,j) \in E_L} J_{ij} s_i s_j, \quad s_i \in \{-1, +1\}.$$

An embedding assigns each logical variable i a nonempty connected branch set C_i in the realized active hardware graph. A returned embedding must satisfy all of the following:

1. every branch set is nonempty, connected, and uses only active hardware;
2. branch sets are pairwise disjoint at return time;
3. every nonzero logical coupling has at least one physical contact between its branch sets;
4. the physical-qubit cap and all prospective work caps are satisfied;
5. the compiled program preserves logical field and coupling sums under the registered compiler.

Search states may be incomplete or overlapping within the O1 envelope. Search legality, embeddability, return validity, and terminal success are different predicates. The neural model can rank only actions that the environment has already materialized and masked as legal. It cannot create validity by prediction.

The registered endpoint is IF-Q3-S0. A frozen program-feature selector chooses one strength from $(0.5, 1, 2, 4)S_I$ without observing a ground energy, planted solution, sample count, or evaluator result. For a valid terminal embedding ϕ , its selected-strength quality is

$$Y_\psi(\phi) = p_{j_\psi(\phi)}(\phi),$$

where p_j is the majority-decoded probability of reaching the certified logical ground energy within the registered tolerance. Failure-aware utility is

$$U = A Y_\psi(\phi),$$

where $A = 1$ only for a freshly validated returned embedding. Ordinary initializer, search, budget, timeout, compilation, or invalid-return failures remain in the denominator with $U = 0$. Integrity failures such as corrupted artifacts or missing reads abort the experiment instead of being converted to zero.

Physical-qubit count, maximum chain length, connectivity measures, native work, latency, memory, and read count are hard constraints, Pareto axes, or explanatory metrics. More qubits can sometimes improve the sampling landscape, but qubit use is never a positive reward by itself.

System architecture

The runtime separates learned preference from exact mechanics.

Layer	Responsibility	Forbidden shortcut
Data authority	Authenticate CandidateBank, lineage design, evaluator targets, and ground certificates	Self-signed corpus used as scientific authority
Classical initializer	Produce the Profile-I starting embedding under prospective work limits	Uncharged restarts or hidden fallback
Exact environment	Own state, candidate materialization, action masks, overlap, archive, transitions, and work	Learned validity or post-selection candidate generation
IF-Core	Encode graphs and bound actions, then produce a masked policy and critics	Access to witnesses, optimum energy, or evaluator outcomes

Layer	Responsibility	Forbidden shortcut
Strength selector	Choose one terminal program strength from deployment-visible features	Four-strength oracle selection at deployment
Independent evaluator	Compile, sample, decode, and score fresh terminal reads	Reuse of training reads for confirmation
Evidence layer	Bind raw rows, sidecars, receipts, identities, and aggregate rules	Success-only filtering or mutable result folders

The classical ordering, placement, routing, and repair stages remain represented. Ordering appears in proposal priorities and construction actions. Placement and routing are enabled in the construction profile. Repair and refinement are the main Profile-I action families. The learned policy replaces preference decisions inside this flow, while exact graph and programming checks remain external to the model.

State, actions, and work

A decision state contains the logical instance, active host, immutable objective context, relaxed branch sets, full ownership claims, valid archive, search memory, remaining nine-coordinate work vector, and a fully materialized candidate batch. Actions bind both old and new memberships. Group rewrites are applied atomically so a legal coordinated move cannot be rejected because of an arbitrary sequential ordering.

The finite action grammar contains `PLACE`, `ROUTE`, `REWRITE`, `GROUP_REWRITE`, `REPAIR`, `RESTART`, `COMMIT`, and the registered stop behavior. Profile I emphasizes rewrite, repair, restart, restore, and commit. The action cap is 64 state-changing candidates, eight archive commits, and one optional stop. The archive contains one protected initializer plus seven FIFO entries.

Candidate construction is outcome-blind. It mixes route length, occupancy pressure, contact multiplicity, local free neighborhood, bottleneck features, and fixed random perturbations. All attempted routes, rejected proposals, materialization, validation, archive operations, and optional cut features are charged before the actor chooses. C++ checks every prospective work delta and returns the exact non-exceeding ledger with the exhausted coordinate.

IF-Core model architecture

IF-Core is a typed graph actor-critic with hidden width 128. It is deliberately compact enough to support controlled comparisons.

1. Three dual local blocks process logical and hardware graphs separately.
2. Two typed fusion blocks exchange information through ownership and conflict relations.
3. Phase-aware factor encoders preserve OLD, NEW, and ARCHIVE chain identities.
4. A two-layer gated one-dimensional convolution encodes ordered routes.
5. Action encoders combine descriptors, chain factors, routes, conflicts, and archive references.
6. Mean and maximum segmented pooling produce state and action-set summaries.
7. A masked categorical actor scores only the exact support.
8. A utility critic estimates failure-aware terminal return. A separate failure critic is active in construction experiments.

Core tensor widths are fixed: logical nodes use 20 value slots plus knownness bits, hardware nodes use 18 plus knownness bits, global context uses 32 plus knownness bits, and action descriptors use 24 plus knownness bits and opcode encoding. The implementation batches graph trunks as a disjoint union, then slices observations before local factor, route, archive, action, and categorical reductions. Environment collection remains sequential because the next support depends on the selected predecessor action.

The model ladder is IF-MLP, IF-Dual, and IF-Core. IF-MLP tests whether hand-engineered action features suffice. IF-Dual tests separate logical and hardware message passing without full typed fusion. IF-Core tests ownership, conflict, and joint-action relational structure. Optional cut tokens, global relay attention, recurrent memory, and learned proposal generation require separate mechanism cells and cannot be introduced silently.

Learning objective and optimization

Quality-v7 rows estimate $Q_U^\mu(s, a)$ under a named frozen continuation policy μ . The semantic target remains `if-q3-s0-qmu-5`. They retain the complete action support, exact observation, continuation seeds, terminal outcomes, and independent evaluator counts. The supervised warm start uses a simultaneous plausible-best set B_s rather than forcing a strict winner when finite-read uncertainty cannot resolve a tie:

$$L_{\text{sup}} = -\frac{1}{|\mathcal{S}|} \sum_s \log \sum_{a \in B_s} \pi_\theta(a \mid s).$$

PPO uses complete episodes, generalized advantage estimation, exact action-support replay, and episode-sum reduction. For ratio $r_t(\theta)$ and advantage $\hat{A}_t$,

$$L_{\text{clip}} = -\mathbb{E} \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The joint loss adds utility-value error, the construction failure-value term when enabled, and the registered entropy schedule. Dropout and stochastic tensor augmentation are disabled during PPO so an unchanged policy replays likelihood ratio one. Gradients never pass through old behavior outputs, environment validity, proposal generation, or the frozen selector.

Reference values include a 32-decision horizon, 64 complete episodes per rollout, four PPO epochs, 256-transition minibatches, $\gamma = 1$, GAE $\lambda = 0.95$, clip 0.2, AdamW learning rate 3×10^{-4} , gradient cap 0.5, and full-buffer KL stop 0.02. The three scientific seeds are 1103, 2207, and 3301.

Data preparation and authority

EmbedBench is an independent package. IsingFold consumes its authenticated CandidateBank-v2 export; it does not fork the generator into the training runtime. The export combines feasible ink-drop witnesses on Chimera, Pegasus, and Zephyr, realized fault maps, exact structural motifs, planted frustrated-loop Ising instances, application-derived instances, and replayable quality records. An embedding witness certifies feasibility only. A planted logical certificate establishes ground energy only. Neither identifies a quality-optimal embedding.

The production importer also requires an independently pinned corpus-design v2 manifest. It publishes prepared schema v4 and checks:

- exact quotas over immutable base lineages;
- at least 1,024 train, 512 validation, and 1,546 sealed-test base lineages;
- application-derived and synthetic origins, multiple host families, and faulted conditions;
- separate embedding, sampling, and decision-quality difficulty axes;
- hard and OOD coverage that is fixed without method outcomes;
- confirmatory power and precision targets covering the complete realized test population;
- validation-only baseline-tuning targets covering the complete realized validation population.

The validation tuning design requires at least 128 independent lineages. Its registered paired valid-return calculation gives 127 under one-sided alpha 0.05, power 0.8, discordance 0.1, true difference 0.05, and margin 0.02. Its bounded paired-utility precision calculation gives 97 for a two-sided 95 percent half-width of 0.2. Both are rounded up to the prospective floor of 128.

Prepared v4 physically separates evaluator targets into `train`, `val`, and `test` files and puts only their hashes, counts, and set digests in the public manifest. Every target-opening command requires an out-of-band publisher ID and attestation record digest. Publisher attestation v2 binds the prepared manifest, partitioned target authority, and separately hashed evidence manifests. `verify-ground-certificates` executes a separately pinned standalone verifier under protocol v2. It writes three partition receipts first and publishes `root.json` last. The root contains only commitments and census metadata. Loading it cannot expose a target. A downstream stage opens exactly one authorized partition and retains both its `TargetAccessReceipt` and ground-partition receipt.

Selector labels and quality labels are distinct. Selector data contain complete four-strength count blocks on train and calibration partitions. Quality-v7 data contain exact-replay action counterfactuals. Long quality jobs use deterministic whole-lineage shards. The merger accepts every registered shard exactly once, rejects

gaps and overlap, replays all rows, recomputes denominators, and publishes a canonical corpus. Quality preflight requires at least 128 resolved rows and 128 resolved independent lineages. No single shard may enter training.

Model selection and training

Four gates precede architecture selection:

1. exact graph, programming, structural-label, and certificate conformance;
2. measurable candidate-support quality headroom;
3. frozen strength-selector discrimination against fixed-strength and random controls;
4. adequate valid returns and a nonsaturated utility signal.

The main grid contains exactly (9+18) cells.

Stage	Families and methods	Seeds	Cells
Representation	IF-MLP, IF-Dual, IF-Core with supervised training	3	9
RL value	selected simpler family and IF-Core crossed with supervised-only, PPO warm-start, PPO from scratch	3	18

Validation selection weights every registered seed and immutable base lineage equally, applies the valid-return noninferiority gate against `return_initial`, then orders survivors by unconditional IF-Q3-S0 utility and online cost. The test partition cannot participate. The capacity control is outside the grid: if IF-Dual is selected as the simpler family, a width-128 nine-local-block IF-Dual is compared with IF-Core at all three seeds within 0.1 percent parameter-count difference.

PPO training does not reuse the hand-authored CandidateBank incumbent as if it were a deployment sample. For each training seed, a target-free initializer-bank plan fixes a lineage-equal episode schedule, public task identity, exact LAC runtime and complete-system config. Each conditional episode has a deterministic finite sequence of initializer draws. Generation stops at the first success, retains failed-draw work, and fails closed if the draw cap is exhausted. The bank is sealed only when every scheduled conditional episode resolves. PPO attaches train targets only after the bank is authenticated. This makes the policy’s training distribution explicitly conditional on a successful deployment initializer without hiding initializer failure in the final estimand.

After the RL-value receipt is frozen, complete-system confirmation retrains all three seeds from fresh initialization with no resume. It evaluates every sealed task and repetition from the pre-initialization denominator. Raw terminal evidence, outcome projections, runtime source digests, model identities, selector identity, quality authority, target-access receipts, ground-partition receipts, and work ledgers are authenticated before the crossed seed-by-lineage interval is computed.

Baselines and evaluation

Same-support controls include `return_initial`, random masked choice, resource-lexicographic choice, classical quality-aware ranking, IF-MLP, IF-Dual, supervised IF-Core, and PPO variants. They isolate proposal headroom, representation value, and sequential-policy value.

Stock minorminer is a separate whole-system arm. A finite registry contains its explicit default and time-saturating resource-ranked and frozen-selector quality-ranked variants. Every candidate is evaluated on the complete validation population at all three registered seeds under one machine identity per seed. One immutable receipt freezes the candidate, not a favorable seed, before test access. The test arm then shares the sealed pre-initialization census, pair keys, seed schedule, selector, evaluator reads, prospective caps, and total online wall-clock envelope with the learned arm. Solver-specific internal work coordinates are disclosed rather than claimed equal.

The publication comparison uses a fixed-sequence familywise rule. It first tests valid-return noninferiority at margin 0.02. Utility superiority is confirmatory only if noninferiority passes, the paired unconditional IF-Q3-S0 lower confidence bound exceeds zero, and sample-size and precision guards pass. Conditional quality, qubit use, chain lengths, connectivity, work, latency, memory, and failure reasons are secondary analyses.

Long complete-system and stock-tuning evaluations use one resumable protocol for three workflows: learned confirmation, tuned-stock confirmation, and external validation tuning. A plan commits to the full public population, run coordinates, seed derivation, execution contract, and lineage shards before a target partition is opened. Each shard owns whole immutable base lineages and keeps the same identity-derived seeds as an unsharded run. The shard receipt carries the partition target access, ground authority, compute class, actual node provenance, raw receipts, outcomes, and terminal evidence. The merger requires the complete nonoverlapping key census and recomputes the unsharded sufficient statistics. Publication runs use `pinned-venv` on Apollo or `apptainer` on Goose; `bare-metal` is diagnostic-only.

The final four-strength diagnostic is sealed after both arms and all three seeds are frozen. Its preregistered config is `configs/final_strength_audit_v1.json`; the registry pins both its record digest and file SHA-256. It fixes at least 128 base lineages, no more than 16 lineages per signed stratum, 4,096 reads per block, 20,000 bootstrap replicates, and 128 shards. An outcome-blind plan is sealed before test outcomes. All six source runs are authenticated before the execution manifest is sealed, and that manifest is sealed before audit reads begin. For every valid return, block A independently samples all four strengths and selects the empirical oracle with the lowest index tie rule. Block B samples all four strengths again and estimates oracle-minus-deployed regret. Invalid returns remain in coverage and worst-case sensitivity denominators. Deterministic shards partition complete opportunity keys; the merger requires the full nonoverlapping census, reauthenticates source evidence, restores canonical order, recomputes summaries, and publishes atomically. This audit cannot alter training, model selection, baseline tuning, or the primary endpoint, and it is not primary evidence for learned-method superiority.

Reproducibility and HPC execution

The compatibility chain is:

1. prepared-v4, publisher attestation v2, and ground-certificate protocol v2 with a target-free root and partition receipts;
2. selector-label manifest v4 and selector bundle v4;
3. quality record v7, shard/merge v5, and quality preflight v2;
4. release gate v5, representation selection v4, and RL-value freeze v4;
5. checkpoint v3 and training-run receipt v4;
6. learned complete-system receipt v3, evaluation report v4, and aggregate v2;
7. external tuning run/selection v2, external complete-system receipt v3/report v4, and paired aggregate v4;
8. resumable evaluation plan v2, shard receipt v3, and merge receipt v2;
9. final-strength config v1, sampler identity v1, plan v2, row v1, execution manifest v2, shard v1, merge v1, and audit receipt v2.

All scientific outputs are canonical finite JSON, self-digested, written atomically, immutable by default, and checked against caller-supplied file pins where the trust boundary requires one. Runtime identity includes loaded native-extension bytes and imported Python sources.

Apollo has no Slurm, so its launchers run directly. Goose computation runs only through Slurm and uses `srsl` inside an allocation. Seed-to-host assignments remain fixed between training and validation. Distributed initializer generation, labeling, evaluation, and auditing partition immutable whole keys, never partial evaluator blocks. Publication evaluation binds the cluster, scheduler, Slurm partition where applicable, CPU/GPU identity, threads, deterministic setting, runtime or environment image, and source digests.

Claim boundary

The architecture establishes exact interfaces and falsifiable tests. It does not establish that a checkpoint has trained successfully, that the learned method beats minorminer, that simulator gains transfer to a QPU, or that IsingFold dominates every graph family. Claims advance only through the registered gates, fresh three-seed confirmation, complete failure-aware denominator, strong baseline, and independently auditable receipts.

The detailed model equations and tensor slots are in the model specification. Exact execution commands are in the training operations guide. The modular architecture is under the `IsingFold_Architecture_Rev2` directory, and stock-baseline tuning has its own specification in this directory.